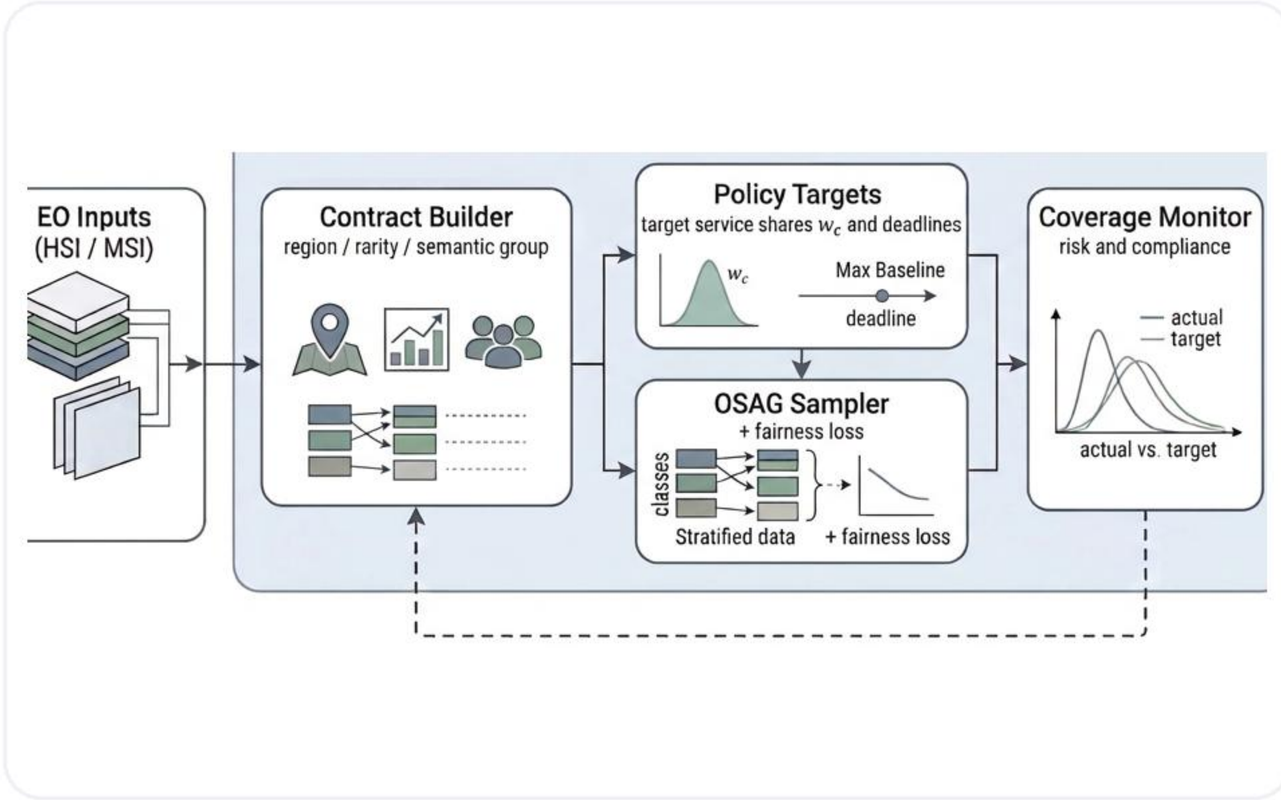
Governance-Aware Earth Observation Learning: Contract-Governed Training with Observed Service Agreement Graphs

Formal English defense presentation based on the locked thesis state

Wenzhang Du
Bachelor of Engineering in Computer Engineering
Faculty of Engineering and Technology
Mahanakorn University of Technology
2026 Thesis Defense

Governance-aware EO training with explicit contract-level service control.



Pipeline visual used in the final locked thesis

Advisor Signature: _____

Talk Roadmap

This 15-minute defense moves from the problem setting to the method, then to fresh evidence, and finally to the dispatch demo and claim boundaries.

1. Problem and Motivation

Why accuracy-only EO training misses the service-allocation question.

2. OSAG Framework and Benchmarks

Contracts, target shares, PrioCovErr, and the HSI/MSI evaluation setup.

3. Evidence: Results, Ablation, and Robustness

Fresh five-seed results, trade-offs, contract design, runtime, and scoped backbone checks.

4. Dispatch Demo, Limitations, and Reproducibility

Operational behavior, claim boundaries, and the appendix-backed evidence chain.

The roadmap is short on purpose: it previews the flow without adding a separate personal-introduction slide.

Background and Motivation

Why the standard training view is incomplete

- Standard EO training usually optimizes aggregate predictive utility such as overall accuracy.
- Real EO services care about who receives attention during training: rare crops, vulnerable regions, or policy-critical semantic groups.
- If service allocation stays implicit, a model can look strong globally while underserving the contracts that matter most.
- This thesis treats training exposure as a governance object rather than as a side effect of data frequency.

Problem in one sentence

The key question is not only 'Does the model perform well on average?' but also 'Who is being served during training?'

Why governance is needed

Accuracy-only view

good global score

Can still ignore policy-critical strata if they are statistically small.

Governance view

explicit service

Tracks whether observed exposure matches the intended contract policy.

This thesis adds a lightweight governance layer instead of replacing the classifier backbone.

Research Gap and Guiding Questions

Central gap

EO training pipelines rarely expose service allocation explicitly. They optimize losses over samples, not compliance with a policy over contracts.

RQ1

How can EO training be reformulated so that service allocation over meaningful contracts becomes explicit and measurable?

RQ2

Can a lightweight governance layer drive empirical coverage toward target shares without requiring a new backbone?

RQ3

What is the empirical relationship between governance quality and predictive utility on HSI and MSI benchmarks?

RQ4

Can the thesis provide a runnable demo that communicates the governance mechanism clearly in a defense setting?

The deck follows the same arc: problem, method, fresh evidence, operational demo, and clear claim boundaries.

Thesis Contributions

Conceptual

Reframes EO training as a service-allocation and governance problem rather than an accuracy-only optimization problem.

Methodological

Develops OSAG as a contract-aware governance layer with target service shares, empirical coverage tracking, alpha mixing, and lambda_C fairness regularization.

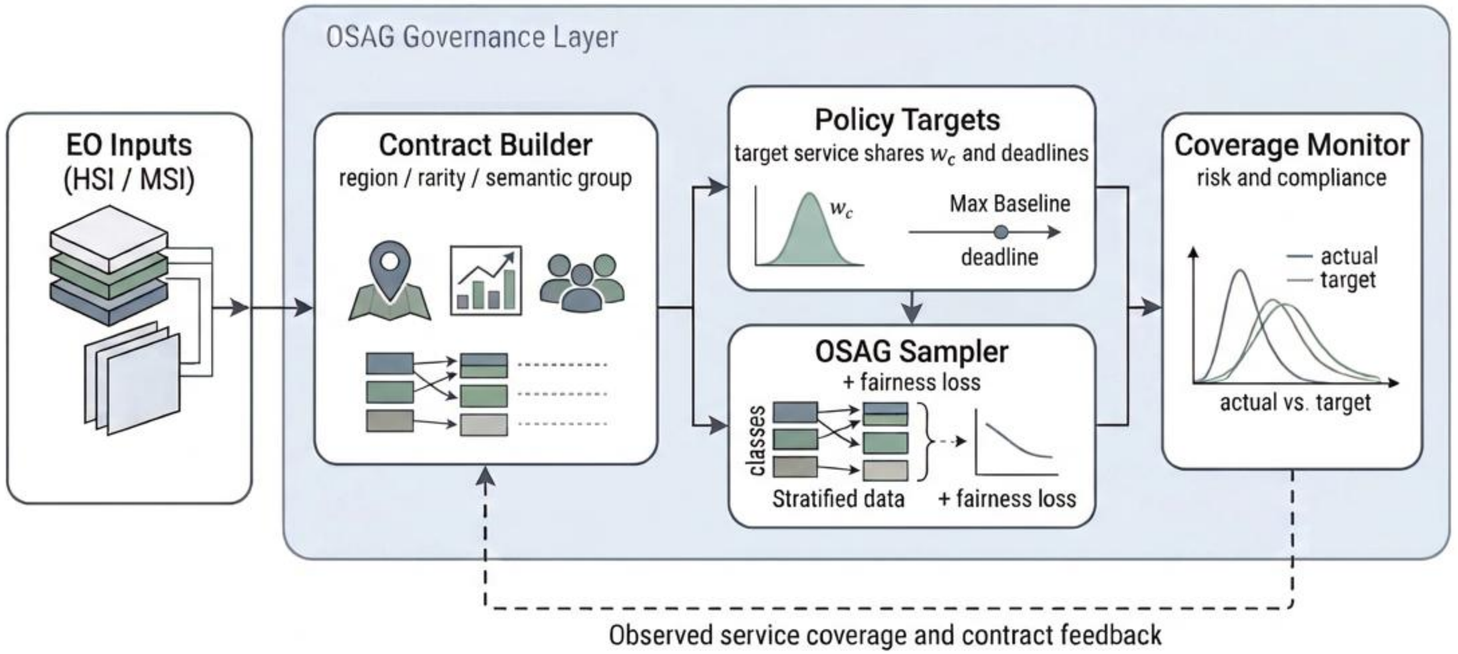
Empirical

Provides fresh five-seed reruns on corrected Indian Pines + Salinas and EuroSAT MSI, plus contract ablation, runtime, and a scoped ResMLP robustness extension.

Artifact-level

Adds a dispatch demo and a reproducibility appendix so the thesis can be defended through both benchmark evidence and a runnable operational artifact.

Figure 1.1: Method Overview



How to read the pipeline

- EO inputs provide sample metadata, labels, and contract-relevant attributes.
- The Contract Builder groups samples into service units such as grid cells, semantic groups, or rarity-aware groups.
- Policy Targets convert priorities and contract sizes into explicit target service shares.
- The OSAG Sampler injects the policy into training exposure, while the Coverage Monitor measures compliance.
- The feedback loop makes service allocation visible, auditable, and controllable during ordinary training.

Key point: OSAG wraps a standard EO classifier. It does not replace the backbone.

Why Ordinary EO Training Is Service-Blind

Standard ERM view

$$\min_\theta \frac{1}{N} \sum_i \ell(f_\theta(x_i), y_i)$$

This objective treats samples as a flat pool. It does not know which contract is mission-critical, which region is vulnerable, or which semantic group should receive more training attention.

Why that becomes a problem in EO

- Large or densely labeled strata automatically receive more optimization exposure.
- Rare but policy-critical contracts can stay under-served even when overall accuracy is competitive.
- The training process has no built-in notion of who should be served in what proportion.

What the governance layer changes

- First, define meaningful contracts instead of treating the dataset as one undifferentiated pool.
- Second, assign target service shares that represent the intended training policy.
- Third, monitor empirical coverage so policy misalignment becomes measurable as PrioCovErr.
- OSAG then turns this governance logic into a lightweight sampling-and-monitoring wrapper around ordinary EO training.

Core Concepts

Contract

A meaningful service unit, not just a class label. Example: dataset + grid cell + rarity flag.

Target service share

The intended fraction of training attention assigned to each contract.

Empirical coverage

The observed fraction of training steps actually spent on each contract.

PrioCovErr

The L1 distance between empirical coverage and target shares. Smaller means better policy compliance.

alpha

Mixing knob between the baseline sampler and pure OSAG. Higher alpha means stronger governance control.

lambda_C

Fairness-loss weight that adds pressure when high-priority contracts remain harder in loss space.

Benchmarks and Data Sources

Hyperspectral benchmark

- Corrected Indian Pines and corrected Salinas are aligned to a common spectral subset.
- The two scenes are merged into one joint HSI governance benchmark.
- Formal inputs come from canonical local files under dataset/: Indian_pines_corrected.mat, Indian_pines_gt.mat, Salinas_corrected.mat, and Salinas_gt.mat.
- The current rerun uses a class-stratified 60/20/20 split after pixel flattening.

Multispectral benchmark

- EuroSAT is used in its canonical 13-band MSI form, extracted from dataset/EuroSAT_MS.zip.
- This benchmark gives a second sensing modality and supports coarse-versus-fine contract design.
- It is also the benchmark used for the scoped ResMLP robustness extension.
- The same formal rerun pipeline regenerates logs, summary tables, and runtime records from local inputs.

These are the authoritative inputs behind the thesis evidence chain. No community mirrors or legacy notebook-only runs are used as the main benchmark evidence.

Contract Construction and Target-Share Policy

How contracts are built

Case	Schema	Count
HSI	dataset + 4x4 grid cell + rare-class flag	44
EuroSAT coarse	four semantic service groups	4
EuroSAT fine	semantic group + training-split rare flag	6

Target-share rule

Target share is proportional to priority times contract size:

$$w_c = (p_c * n_c) / sum_{c'} (p_{c'} * n_{c'})$$

This keeps policy grounded in both importance and available evidence.

Priority assignments in the locked rerun pipeline

- HSI: rare contracts receive priority 3; non-rare contracts receive priority 1.
- EuroSAT coarse: urban and water receive priority 2; agri and natural receive priority 1.
- EuroSAT fine: rare contracts receive priority 2; non-rare contracts receive priority 1.
- This is why contract design is not just preprocessing. It directly determines what governance target the learner is asked to follow.

Repository and Code Architecture

Real benchmark rerun path

- scripts/run_all_real_experiments.py is the one-click entry for the formal rerun pipeline.
- scripts/reproduce_osag_real.py contains contract construction, training, fairness-loss logic, and coverage monitoring.
- scripts/generate_real_result_assets.py turns fresh CSV outputs into final tables and figures.

Dispatch demo path

- osag_demo/run_visual_demo.py delegates to run_dispatch_demo.py.
- dispatch_demo_core.py simulates contracts, budgets, deadlines, and policy-driven observation decisions.
- dispatch_demo_assets.py writes the dashboard, timeline plot, step comparison, and contract-gap visualizations.

Presentation and artifact path

- deliverables/ stores the official defense deck and preview files.
- reports/ stores notes, audit files, code walkthroughs, FAQ, and final package reports.
- The public repository mirrors the final thesis PDF, the official deck, and the defense-day support files.

Defense reading point: the software story is organized around a rerun path, a demo path, and an artifact-generation path. That makes code questioning much easier to handle.

Code View: Contracts and Target Service Shares

reproduce_osag_real.py / build_contract_table_from_meta

```
meta['contract_key'] = meta[key_cols].astype(str).agg('|'.join, axis=1)
grouped = meta.groupby('contract_id', sort=True)
num_samples = grouped.size().rename('num_samples')
priorities = grouped.apply(priority_rule, include_groups=False)
df_contracts['raw_weight'] = df_contracts['priority'] * df_contracts['num_samples']
df_contracts['target_weight'] = df_contracts['raw_weight'] / df_contracts['raw_weight'].
```

What this means

- Contracts are materialized as IDs, sample counts, priorities, and target weights.
- The target-share policy is exactly the thesis rule: priority times contract size, normalized across contracts.
- That is the bridge from the mathematical policy definition to an actual training-time object used by the sampler and monitor.

dispatch_demo_core.py / CONTRACTS

```
CONTRACTS = [
  {'contract_name': 'Medical Core', 'priority': 3, 'target_weight': 0.22, 'deadline_steps': 1},
  {'contract_name': 'Industrial Fire Belt', 'priority': 3, 'target_weight': 0.20, 'deadline_steps': 1},
  {'contract_name': 'Evacuation Corridor', 'priority': 3, 'target_weight': 0.18, 'deadline_steps': 2},
  {'contract_name': 'Residential South', 'priority': 2, 'target_weight': 0.16, 'deadline_steps': 2},
  {'contract_name': 'Floodplain East', 'priority': 2, 'target_weight': 0.14, 'deadline_steps': 2},
  {'contract_name': 'Perimeter Background', 'priority': 1, 'target_weight': 0.10, 'deadline_steps': 4},
]
```

Code View: OSAG Sampler, Fairness, and Coverage Monitor

dispatch_demo_core.py / choose_observations

```
score[idx] = (
    1.44 * row['urgency']
  + 1.30 * deadline_pressure[cid]
  + 0.86 * coverage_gap[cid] / observe_budget
  + 0.46 * uncertainty
  + 0.44 * priority_norm[idx]
)
chosen = env_df.iloc[order[:observe_budget]]['cell_id']
```

reproduce_osag_real.py / fairness-loss branch

```
if method.method == 'osag_priority_fairloss' and method.lambda_c > 0.0:
    high_mask = prio_b >= 2.0
    low_mask = prio_b < 2.0
    fair_penalty = clamp(mean(losses[high]) - mean(losses[low]), min=0.0)
    loss = loss + method.lambda_c * fair_penalty
```

coverage monitoring

```
coverage_metrics = compute_coverage_errors(contract_counts, ordered_contracts)
observed = contract_counts / max(contract_counts.sum(), 1.0)
priority_coverage_error_mean = |observed - target| aggregated over contracts

Practical meaning:
1. the sampler tries to reduce service gaps during training;
2. the fairness branch adds pressure on hard, high-priority contracts;
3. the monitor makes policy alignment measurable instead of hidden.
```

Code View: Benchmark and Demo Execution Path

Formal benchmark commands

rerun entry points

```
python .\scripts\run_all_real_experiments.py
python .\scripts\generate_real_result_assets.py

Key outputs:
results\tables\rerun_20260321_combined_main5\fresh_main_results.csv
results\runtime\rerun_20260321_combined_main5\runtime_summary.csv
```

- If asked to show the benchmark code, start from run_all_real_experiments.py, then jump into reproduce_osag_real.py.
- That path is the cleanest explanation of how local data become logs, fresh CSVs, tables, and runtime summaries.

Dispatch demo commands

demo entry points

```
python .\osag_demo\run_visual_demo.py
python .\osag_demo\launch_visual_demo.py

Files to open if questioned:
osag_demo\run_dispatch_demo.py
osag_demo\dispatch_demo_core.py
osag_demo\dispatch_demo_assets.py
```

- run_visual_demo.py is intentionally simple. It delegates to run_dispatch_demo.py so the defense entry is easy to remember.
- The safe live order is: generate demo artifacts, open the dashboard, then explain contract selection and timeline behavior from dispatch_demo_core.py.

Main HSI Benchmark Results: Indian Pines + Salinas

Fresh five-seed operating points

Policy	Acc_all	Acc_high	PrioCovErr
Random	91.47	82.97	23.49
Class-Balanced	90.25	86.26	31.37
Uniform-Contract	90.90	96.22	96.05
Contract-Priority	89.04	93.75	107.00
OSAG	90.85	88.83	0.26
OSAG-FairLoss (l=0.5)	91.10	91.15	0.26

What matters here

- Random keeps strong Acc_all but leaves policy misalignment high at 23.49.
- Class-Balanced improves Acc_high, yet actually worsens PrioCovErr to 31.37.
- OSAG collapses PrioCovErr to 0.26 while improving Acc_high to 88.83.

Interpretation

- Heuristic contract baselines can produce very high Acc_high numbers, but they do so by overserving some contracts badly.
- OSAG-FairLoss keeps the same near-zero PrioCovErr and pushes Acc_high to 91.15.
- OSAG-Mix (a=0.50) is also important: it reaches the best HSI Acc_all, 91.68, while already cutting PrioCovErr to 11.75.

HSI takeaway: class rebalancing and contract heuristics are not enough. The governance-aware policies are the ones that align service exposure with the target policy.

Main EuroSAT MSI Results

Fresh five-seed operating points

Policy	Acc_all	Acc_high	PrioCovErr
Random	86.93	73.68	23.81
Class-Balanced	85.91	74.25	17.18
Uniform-Contract	86.58	76.02	9.48
Contract-Priority	86.41	77.03	42.82
OSAG	86.39	76.05	0.20
OSAG-FairLoss (l=0.5)	86.30	77.04	0.20

What matters here

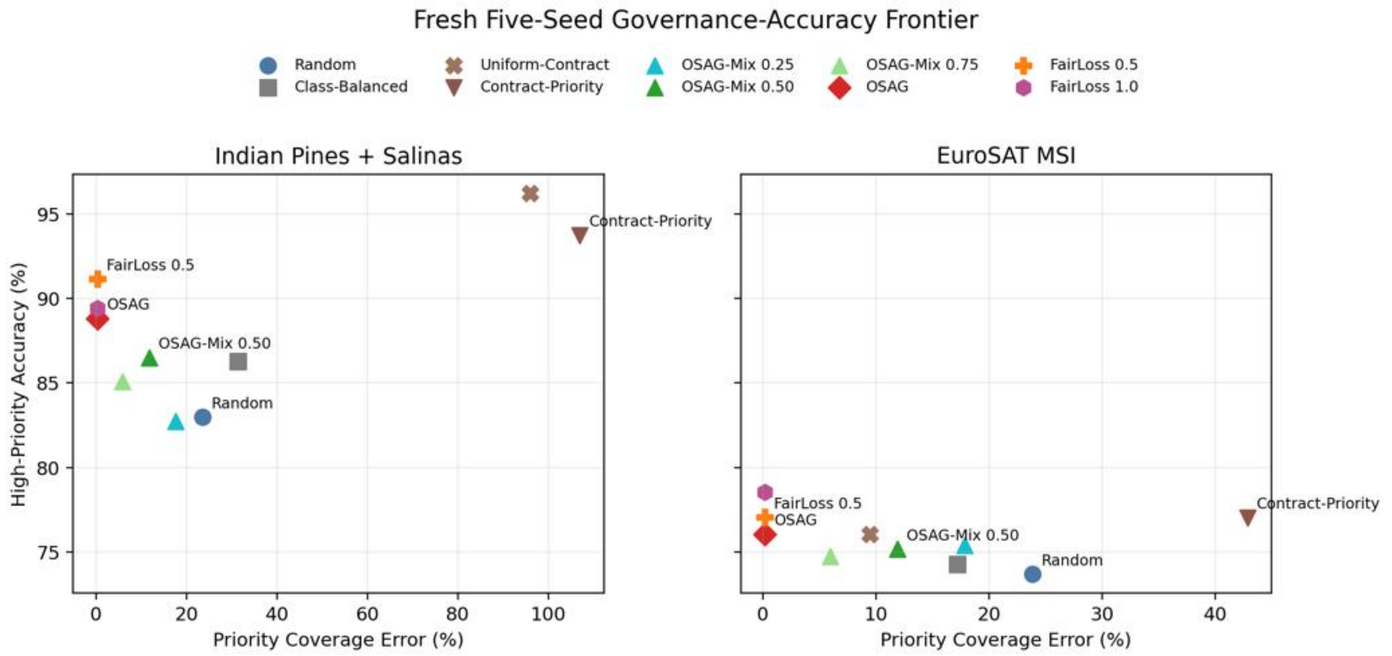
- Random keeps the strongest Acc_all, 86.93, but leaves PrioCovErr at 23.81.
- Uniform-Contract lowers PrioCovErr, but still stays far from explicit policy alignment.
- OSAG drives PrioCovErr down to 0.20 and improves Acc_high to 76.05.

Interpretation

- Contract-Priority reaches 77.03 Acc_high, but its PrioCovErr explodes to 42.82, which means it overserves some priority contracts in a distorted way.
- OSAG-FairLoss (l=0.5) keeps the same near-zero PrioCovErr and raises Acc_high further to 77.04.
- OSAG-Mix (a=0.75) is the softer operating point: Acc_all stays at 86.70 while PrioCovErr already falls to 5.96.

EuroSAT takeaway: the governance effect is not limited to hyperspectral data. The same policy-alignment logic appears on a second modality with a different contract design.

Why Accuracy Alone Is Not Enough



Read Table 5.2 correctly

The reported deltas are absolute percentage-point differences relative to Random, not relative percentage changes.

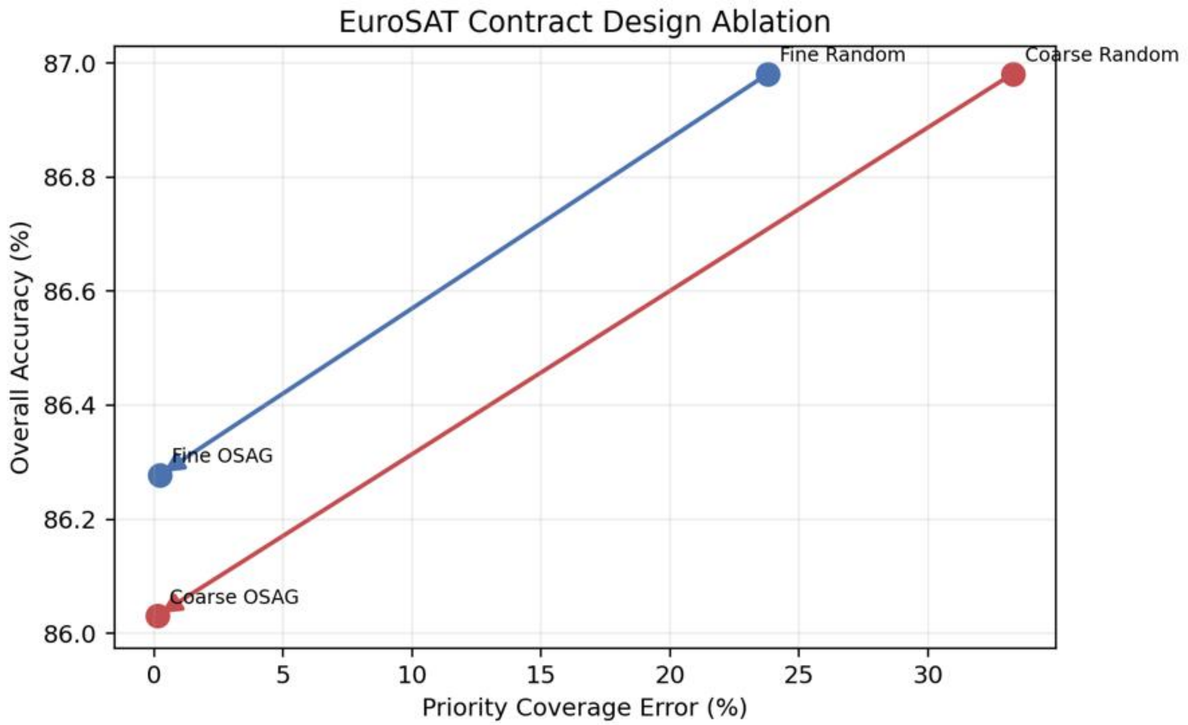
Two benchmark examples

- HSI OSAG vs Random: Acc_all -0.62, Acc_high +5.85, PrioCovErr -23.23 points.
- EuroSAT OSAG vs Random: Acc_all -0.54, Acc_high +2.36, PrioCovErr -23.61 points.
- Heuristics can look good on Acc_high alone, yet still overserve some contracts and violate the target policy badly.

Takeaway

The thesis contribution is governance-aware training, not raw Acc_all domination.

Contract-Design Ablation



Fresh three-seed EuroSAT study

Variant	Rnd Acc_all	OSAG Acc_all	Rnd PrioCovErr	OSAG PrioCovErr
Coarse	86.98	86.03	33.29	0.12
Fine	86.98	86.28	23.80	0.21

Why this matters

- Fine contracts start from a lower Random PrioCovErr baseline.
- Both contract designs can be driven toward near-zero policy error.
- The fine design retains more accuracy while doing so, which means governance cost depends on how the contracts are defined.

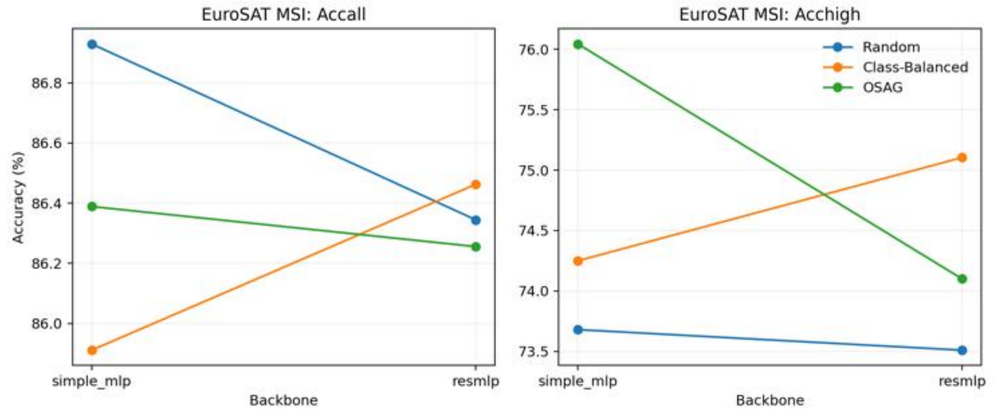
Runtime and Backbone Robustness

Runtime interpretation

Dataset	Policy	Avg/Epoch (s)	Total (s)
HSI	Random	3.524	211.466
HSI	OSAG	2.938	176.285
HSI	OSAG-FairLoss (0.5)	3.035	182.120
EuroSAT	Random	1.367	82.044
EuroSAT	OSAG	1.358	81.509
EuroSAT	OSAG-FairLoss (0.5)	1.462	87.737

- The correct wording is comparable overhead, not speed superiority.
- On HSI, the higher Random mean is driven mainly by one outlier seed rather than by a stable algorithmic advantage for OSAG.
- The fairness penalty is somewhat more expensive, but it remains in the same practical range.

Scoped EuroSAT ResMLP extension



- This is a robustness extension, not a replacement for the main benchmark design.
- The stronger backbone changes absolute utility values slightly.
- The central governance conclusion remains: OSAG still achieves near-zero PrioCovErr under both backbones.

Why the Dispatch Demo Is Needed

What the benchmark chapter already proves

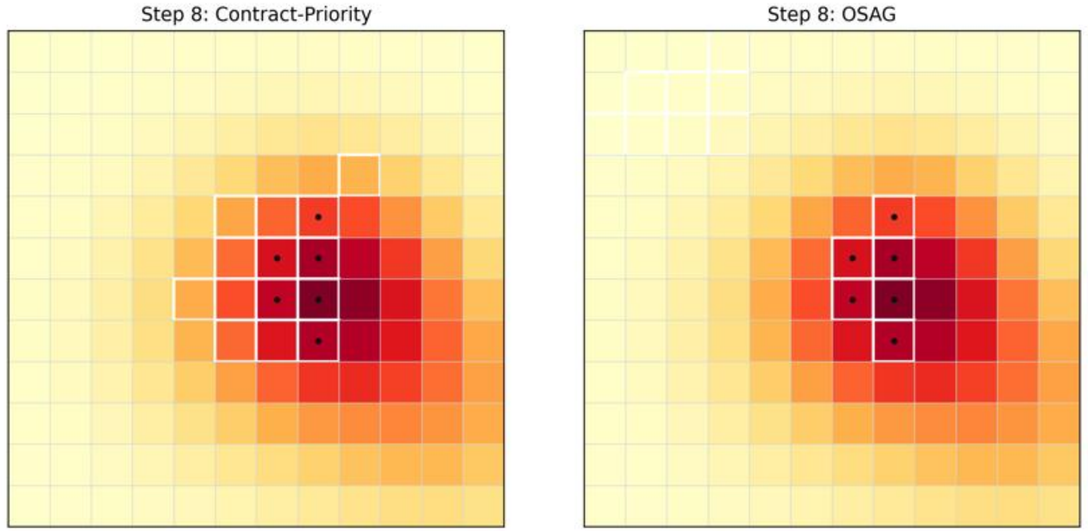
- Fresh reruns on corrected HSI and EuroSAT MSI provide the main scientific evidence.
- Those tables answer whether contract-aware governance changes the benchmark outcome.
- They do not naturally show the time-stepped behavior of budgets, deadlines, and missed service.

What the demo adds

- It turns the same governance logic into a live operational story.
- A committee can see which contracts are served, which ones drift, and how missed-service events accumulate over time.
- The demo is therefore an interpretability and communication artifact, not a substitute for the real benchmark.

The presentation uses the slide deck to explain the dispatch system. The live demo happens after the presentation and is not counted inside the timing estimate.

Dispatch Demo Setup and Scenario



Scenario definition

- 12 x 12 urban emergency grid over 10 decision steps.
- Six contracts: Medical Core, Industrial Fire Belt, Evacuation Corridor, Residential South, Floodplain East, and Perimeter Background.
- The first three are high-priority contracts with short revisit deadlines.

Budgets and compared policies

- Observation budget: 14 tiles per step.
- Annotation budget: 6 tiles per step.
- Policies compared: Random, Contract-Priority, and OSAG.
- The dashboard tracks five quantities: compliance, coverage error, missed service, Q_high, and reliability.

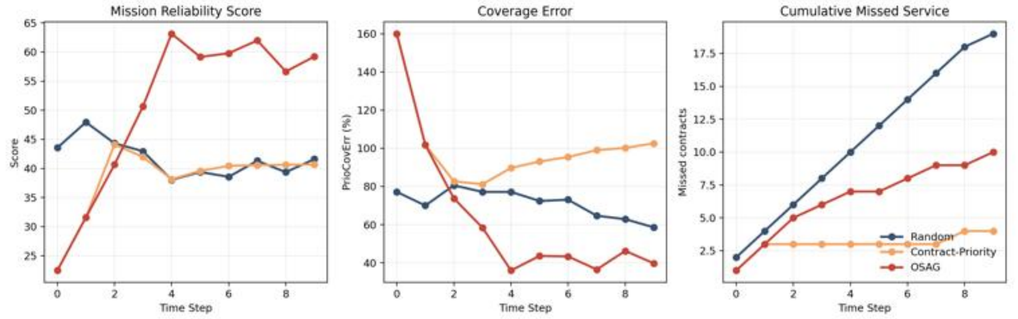
Dispatch Demo Results and Interpretation

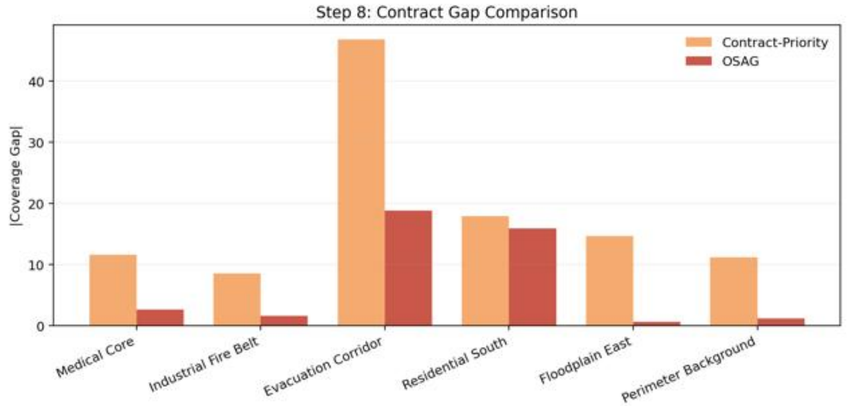
End-of-run outcomes

Policy	Reliability	Compliance	Coverage Err	Missed	Q_high
Random	41.60	53.67	0.586	19	82.68
Contract-Priority	40.68	22.91	1.026	4	80.43
OSAG	59.23	60.96	0.397	10	81.47

How to read the result

- OSAG is strongest on the governance-oriented summary because it balances compliance, coverage control, and service continuity.
- Contract-Priority minimizes missed service more aggressively, but does so by letting long-run service balance drift badly.
- Q_high is a separate quality measure. It is not the same quantity as the critical-capture term K in the reliability equation.





Sensitivity note: OSAG remains top-ranked under equal weights and several local weight perturbations around the thesis setting.

Limitations and Claim Boundaries

What the thesis supports

- Governance-aware sampling can strongly reduce contract-level policy misalignment on the current HSI and EuroSAT MSI benchmarks.
- The effect survives fresh reruns, a contract-design ablation, runtime scrutiny, a scoped ResMLP extension, and an operational demo.
- The reproducibility appendix makes the evidence chain auditable through scripts, configs, manifests, logs, and saved outputs.

What is not being claimed

- The HSI split is pixel-stratified rather than spatial-blocked, so absolute accuracy may be optimistic relative to strict deployment conditions.
- The 'graph' is a modeling view, not a full explicit graph optimizer in the current implementation.
- The main benchmark uses a lightweight MLP to isolate governance effects; the thesis is not a backbone SOTA study.
- The ResMLP extension is scoped to EuroSAT only.
- The demo is supportive evidence, not the main benchmark.
- Provenance is documented through manifests and stage reports rather than a git commit ledger inside the thesis workspace.

Reproducibility and Public Release

Fully rerun main benchmark

HSI and EuroSAT MSI main tables are fresh five-seed reruns from formal scripts and canonical local inputs.

Fully rerun extensions

EuroSAT coarse-vs-fine ablation is a fresh three-seed rerun; the EuroSAT ResMLP extension is a fresh five-seed rerun.

Artifact-level reproducibility

Dispatch demo outputs, thesis figures/tables, and the thesis PDF can all be regenerated locally.

Evidence boundary and companion release

- Historical conference CSV files remain in the workspace only for traceability checks. They are not presented as fresh evidence.
- The thesis workspace is not a git repository, so provenance is documented through manifests, configuration snapshots, saved outputs, and stage reports.
- Companion public source release: https://github.com/dqswordman/osag-eo-governance

Final Takeaways and Transition

1. Core thesis claim

EO training is not only an accuracy problem. It is also a service-allocation and governance problem.

2. Main empirical claim

Fresh reruns show that OSAG sharply reduces policy misalignment while keeping competitive predictive utility.

3. Defense claim

The benchmark, the dispatch slides, and Appendix A together give a complete and auditable defense story.

This concludes the slide presentation. After this, I will move to the live demo and then to questions from the committee.

Backup: Where is the 'graph' really?

- In the current thesis, the graph is a modeling view over the contract space, not a separate graph neural network or explicit graph optimizer.
- The graph intuition matters because contracts can be related through semantic similarity, spatial adjacency, or hierarchical refinement.
- Chapter 3 uses this graph view to motivate why contract design changes governance cost, especially in the EuroSAT coarse-vs-fine study.
- Future work could instantiate the graph explicitly through region adjacency graphs or learned semantic neighborhoods.

Safe oral answer

graph = structured contract space

The current implementation is mainly a contract-aware sampling and monitoring layer.

Do not overclaim

not a full graph optimizer

The thesis deliberately treats explicit graph algorithms as future work.

Backup: HSI Split and Spatial Leakage Boundary

What the pipeline does

- Corrected Indian Pines and corrected Salinas are aligned to a common spectral subset and merged into a joint HSI setting.
- Labeled pixels are flattened first, then assigned by a class-stratified 60/20/20 train/validation/test split.
- This is pixel-stratified, not spatial-blocked.

How to answer the leakage question

- Yes, neighboring pixels from the same original scene can appear in different splits.
- That means absolute accuracy may be optimistic relative to a stricter spatial-blocked evaluation.
- The main thesis claim should therefore be read as a comparative governance result under a fixed benchmark protocol.
- The benchmark still remains useful because all compared policies share the same split and the governance effect is large and consistent.

Backup: Demo Metrics, Q_high, K, and Sensitivity

Metric interpretation

- Reliability is a hand-set composite of compliance, coverage error, missed service, and critical capture K.
- Q_high is separate: it is the final mean high-priority model-quality state, not the same quantity as K.
- Coverage error in the demo is an operational contract-level metric analogous to, but not identical with, benchmark PrioCovErr.
- Reliability is useful as a dashboard summary, but it does not replace the primitive metrics.

Sensitivity audit

- No new simulation was rerun for the sensitivity note.
- The audit recomputed rankings from saved dispatch CSV outputs only.
- OSAG stays top-ranked under equal weights and several local perturbations around the thesis weighting.
- Only more extreme capture-dominated weightings favor Contract-Priority.